

SOFTWARE DESIGN DOCUMENT

Meera Premium Imitation Jewellery E-Commerce Platform

- Design objective:**
Define the application architecture, module boundaries, React component structure, data layer, authentication flow and integration points used to build a maintainable real-world e-commerce experience.

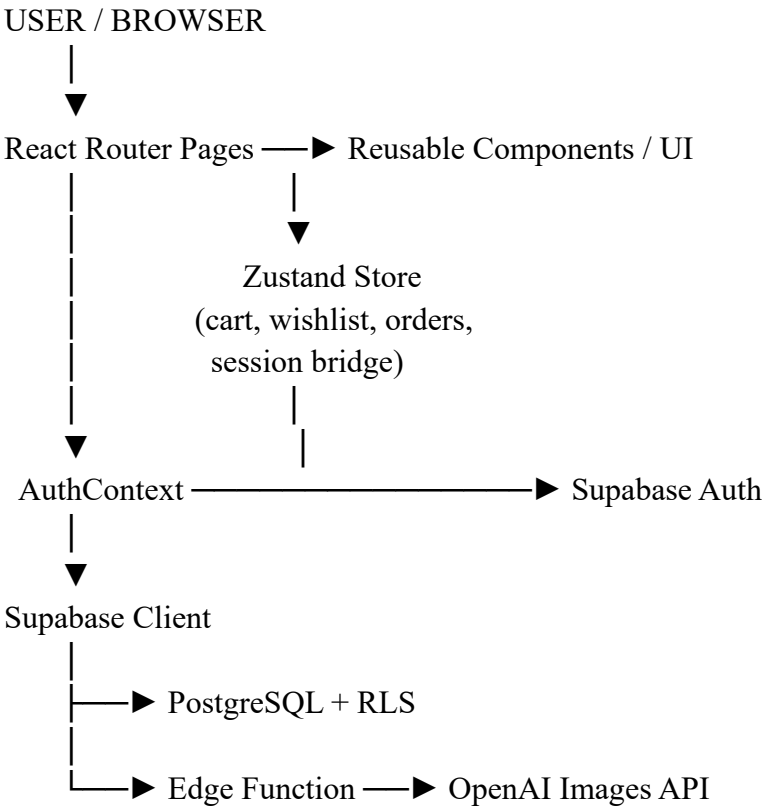
Document Control Details

Item	Details
Document	Software Design Document (SDD)
Application Type	Responsive single-page e-commerce web application
Frontend	React 19 + TypeScript + Vite + Tailwind CSS
State / Data	Zustand + Supabase PostgreSQL
Authentication	Supabase Auth with email/password and OAuth providers
Backend Integration	Supabase client + Supabase Edge Function for AI image generation
Primary Users	Customers and authorized administrator

1. System Architecture

The application follows a component-based SPA architecture. Presentation is separated into reusable React components and route-level pages, while authentication, application state and persistence are handled by dedicated context/store/service layers. Supabase provides managed authentication, PostgreSQL storage and row-level security (RLS).

• **Architecture Flow**



2. Core Technology Decisions

Layer	Technology	Design Rationale
UI Frontend	React + TypeScript	Reusable components, typed interfaces and scalable page structure.
Build	Vite	Fast local development and optimized production bundling.
Styling	Tailwind CSS + custom CSS	Responsive design with reusable utility classes and project-specific visual tokens.
Routing	React Router	Client-side navigation without full page reloads.
State	Zustand + persist	Centralized cart, wishlist, user and order state with persistence.
Backend DB	Supabase	Managed PostgreSQL, authentication, policies and API access without a separate custom server.
AI Integration	Supabase Edge Function	Keeps the OpenAI secret key server-side and validates generation requests.

3. High-Level Design Principles

- **Separation of concerns:** pages handle screen composition, components handle reusable UI, stores/contexts handle state, and service/database layers handle persistence.
- **Reusable UI:** shared navigation, footer, product cards, loading states and Radix-based UI primitives reduce duplication.
- **Security by design:** authenticated data access is scoped with Supabase RLS; administrative access is checked through a dedicated authorization utility.
- **Responsive-first experience:** desktop navigation and mobile navigation are both implemented, with responsive grids and layouts across shopping flows.

4. Functional Module Structure

Module	Responsibilities	Key Files / Components
Storefront	Home, collections, new arrivals, categories and product discovery.	Home.tsx, Shop.tsx, NewArrivals.tsx, category pages
Product Experience	Product detail, image presentation, specifications, wishlist and quick cart actions.	ProductDetail.tsx, ProductCard.tsx
Shopping Cart	Add/remove items, quantity changes, totals and login-required guard.	Cart.tsx, useStore.ts
Checkout & Orders	Address selection, checkout steps, order creation and confirmation/tracking.	Checkout.tsx, OrderConfirmation.tsx, OrderTracking.tsx
Customer Account	Sign-up/login, profile, saved addresses, wishlist and order history.	Login.tsx, Signup.tsx, Profile.tsx, Wishlist.tsx
AI Custom	Collects design requirements and connects to image-generation backend logic.	AICustom.tsx, generate-image Edge Function
Admin	Restricted dashboard for business metrics, products, orders and user-oriented operations.	AdminDashboard.tsx, authorization.ts
Support Content	About, contact, FAQs, shipping, returns, care, size guide, careers and legal pages.	About.tsx, Contact.tsx, FAQs.tsx, etc.

5. React Component & Routing Design

The root App component owns route composition and global layout. Navigation and Footer remain persistent around route-level page components. Feature pages compose smaller reusable components rather than embedding all UI logic in one file.

Project Structure

```
src/
├── App.tsx                # route map + application shell
├── components/            # reusable UI and feature components
│   ├── Navigation.tsx
│   ├── ProductCard.tsx
│   ├── StyleFinder.tsx
│   └── ui/                # reusable Radix/shadcn-style primitives
├── pages/                 # route-level screens
├── sections/              # reusable homepage sections
├── contexts/              # authentication + theme state
├── store/useStore.ts      # client-side business state
├── hooks/useDatabase.ts   # authenticated database operations
└── lib/                   # Supabase, authorization and service helpers
```

6. Key User Journeys

Journey	Design Flow
New customer	Signup → Supabase Auth → profile record → authenticated customer area.
Browse & purchase	Collections → Product Detail → Cart → Checkout → order record → confirmation → tracking.
Wishlist	Product Card → Zustand wishlist → optional Supabase sync for authenticated user.
Saved address	Profile/Checkout → address form → addresses table → default-address selection.
Custom design	AI Custom → validated prompt/style → Edge Function → OpenAI → image generation → UI result.
Administrator	Login → email-based authorization check → Admin Dashboard → business/order views.

7. State Management Strategy

Zustand provides the application-level store for cart, wishlist, customer profile bridge, addresses and orders. The store persists selected client state, while Supabase remains the source of truth for authenticated database records.

AuthContext listens for Supabase session changes and synchronizes the authenticated user into the application store.

8. Database Design

The PostgreSQL schema is designed around authenticated customer ownership. User-specific records reference the profile/user identity and are protected by RLS policies.

JSONB is used for flexible order addresses/items and AI hamper preferences, while indexed user IDs support common lookups.

Table	Purpose	Important Relationships / Fields
profiles	Customer profile extending auth identity.	id auth.users; email, full_name, phone
addresses	Saved delivery addresses.	user_id profiles; default flag; location fields
wishlist	Products saved by a customer.	user_id profiles; product_id; unique user/product pair
contact_submissions	Stores customer enquiries.	name, email, phone, subject, message
ai_custom_hampers	Stores AI-assisted hamper requests.	user_id; preferences JSONB; budget; items JSONB; status
orders	Stores purchase/order records.	user_id; total_amount; status; shipping/billing JSONB; items JSONB

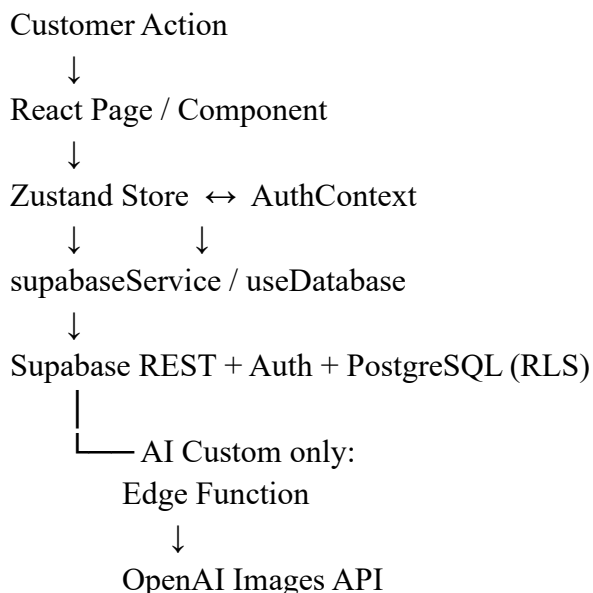
9. Data Access & Integration Layer

Interface	Role in System	Example Operations
AuthContext	Authentication/session boundary.	getSession, signUp, signIn, OAuth, signOut
useDatabase hook	Authenticated database operations.	profiles, addresses, wishlist, contact submissions, custom hampers, orders
supabaseService	Service wrapper used by the Zustand store.	getAddresses, createOrder, getOrders, wishlist operations
Supabase Client	Typed client for PostgreSQL/Auth APIs.	from('orders'), from('addresses'), auth.*
Edge Function	Server-side AI image generation gateway.	POST validation → OpenAI Images API → response

10. Security & Reliability Considerations

- Row Level Security is enabled on **profiles, addresses, wishlist, contact submissions, AI custom hampers and orders**. Policies restrict customer records to the authenticated owner where appropriate.
- Administrative access is isolated through `isAdminEmail()`, so the admin dashboard is not treated as a normal customer route.
- The AI generation Edge Function validates HTTP method, prompt size, allowed style values and uploaded image size before making the external API request.
- Sensitive AI provider credentials are read from server-side environment variables in the Edge Function rather than being embedded in browser-side generation code.
- The frontend uses typed domain models for products, cart items, addresses and orders, reducing data-shape errors during development.

11. API / Data Flow Summary



12. Implementation Status & Engineering Notes

The uploaded project contains a production-oriented React/TypeScript structure, a Supabase schema, authentication context, database helpers, an admin dashboard, customer shopping flows and an AI image-generation Edge Function.

The current checkout UI contains a simulated payment-processing step; a production deployment should replace this with a PCI-compliant payment gateway and server-side payment verification.

Likewise, any remaining client-only fallback state should be synchronized with Supabase before treating the application as a fully transactional production system.

13. Future Scalability

- Introduce a dedicated payment gateway and webhook-based order verification.
- Move product/catalog inventory from static TypeScript data into managed database tables or a headless commerce backend.
- Add automated tests for authentication, cart rules, order creation and RLS policies.
- Add observability: structured logging, error tracking, performance monitoring and audit events for admin actions.
- Use environment-based configuration for Supabase URLs/keys and deployment-specific settings.